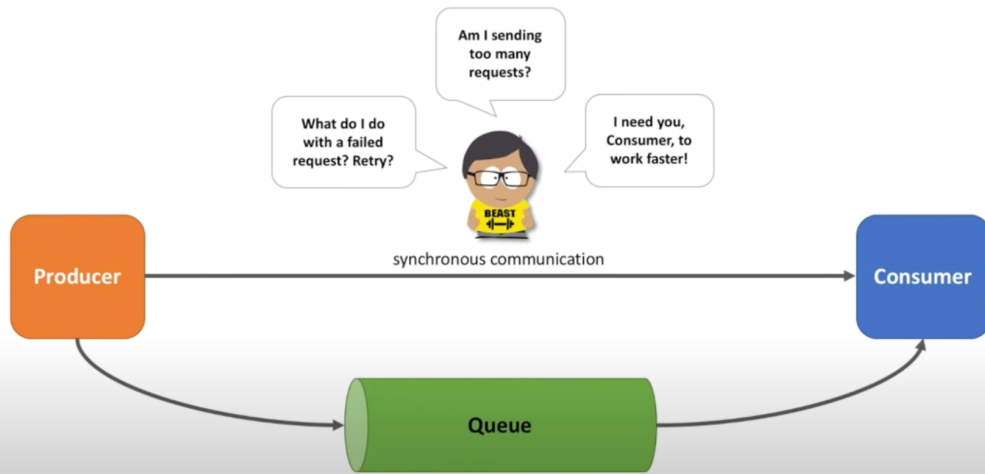


System Design Interview - Distributed Message Queue (Kafka)

 System Design Interview - Distributed Message Queue

Problem Statement



We need to build this

Requirements

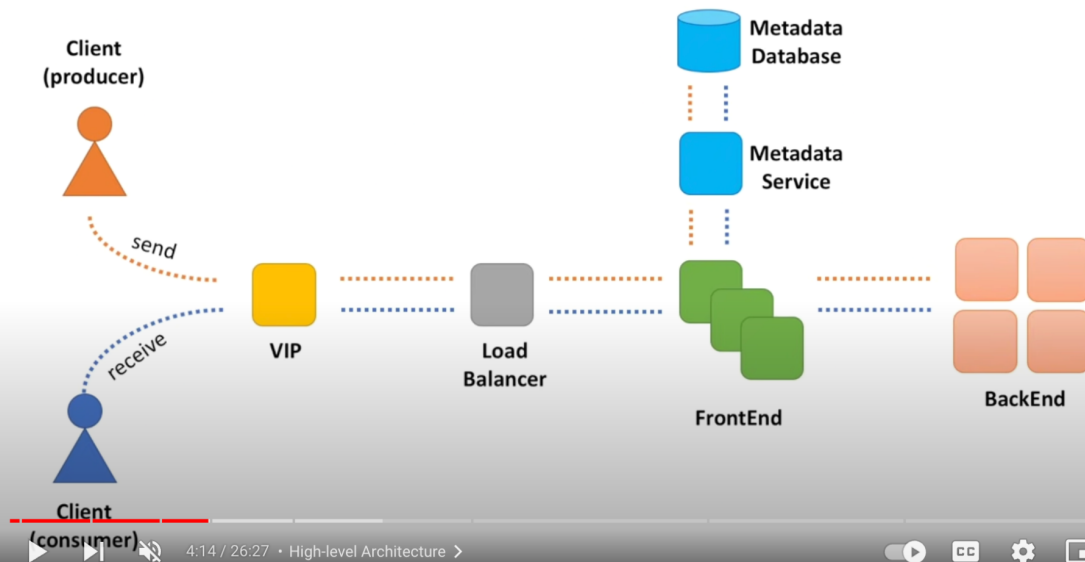
Functional

- `sendMessage(messageBody)`
- `receiveMessage()`

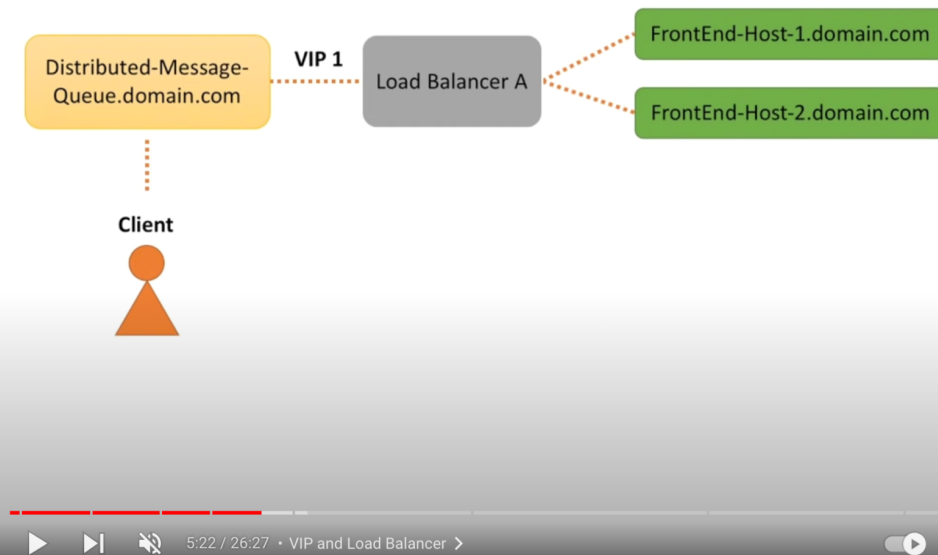
Non-Functional

- **Scalable** (handles load increases, more queues and messages)
- **Highly Available** (survives hardware/network failures)
- **Highly Performant** (single digit latency for main operations)
- **Durable** (once submitted, data is not lost)

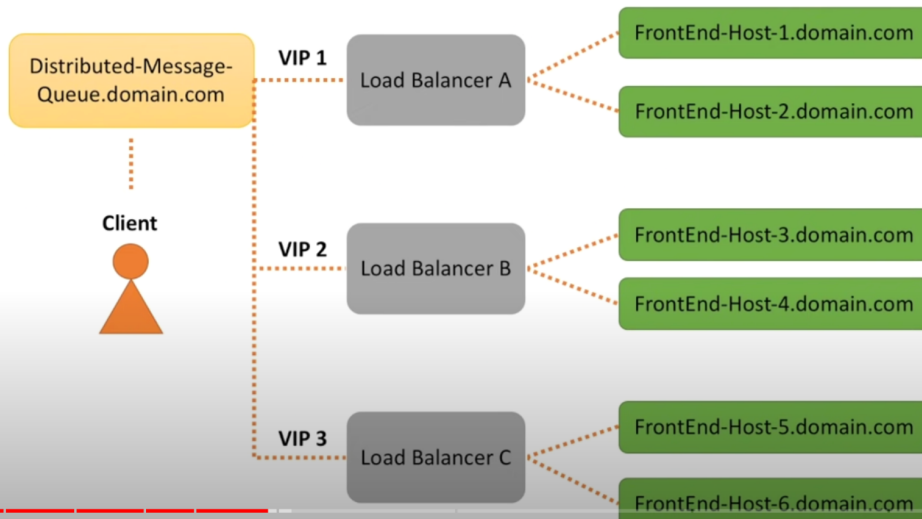
High-level Architecture



VIP and Load Balancer



VIP and Load Balancer



FrontEnd Service

- A lightweight web service
- Stateless service deployed across several data centers

Actions

- Request validation
- Authentication/Authorization
- TLS (SSL) termination
- Server-side encryption
- Caching
- Rate limiting (throttling)
- Request dispatching
- Request deduplication

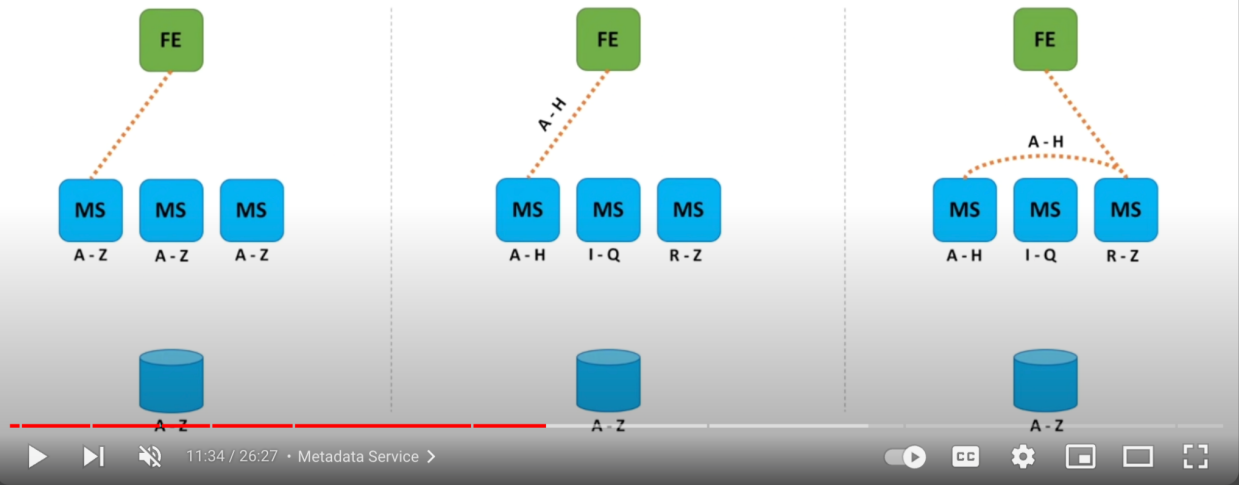
Request deduplication

- May occur when a response from a successful sendMessage request failed to reach a client
- Lesser an issue for 'at least once' delivery semantics, a bigger issue for 'exactly once' and 'at most once' delivery semantics
- Caching is usually used to store previously seen request ids

- Usage data collection

Metadata Service

- A caching layer between the FrontEnd and a persistent storage
- Many reads, little writes
- Strong consistency storage is preferred but not required



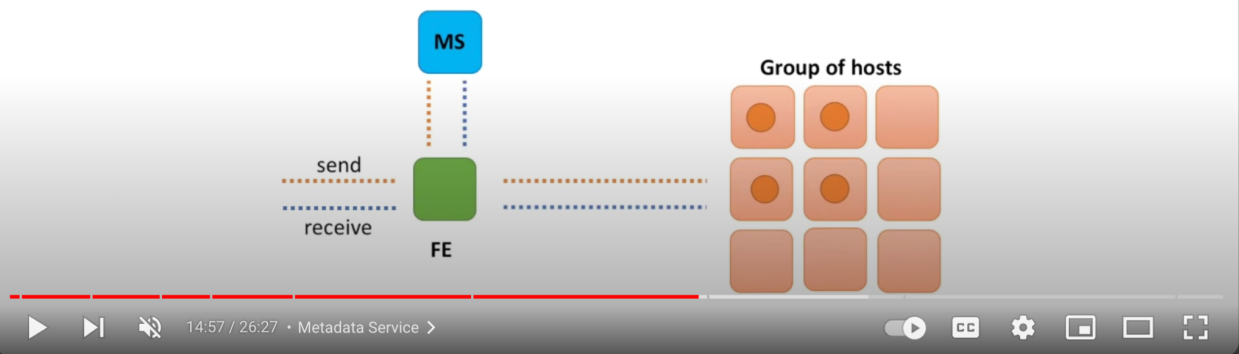
BackEnd Service

- Where and how do we store messages?
- How do we replicate data?
- How does FrontEnd select a backend host to send data to?
- How does FrontEnd know where to retrieve data from?

Answer: RAM and a local disk of a backend host

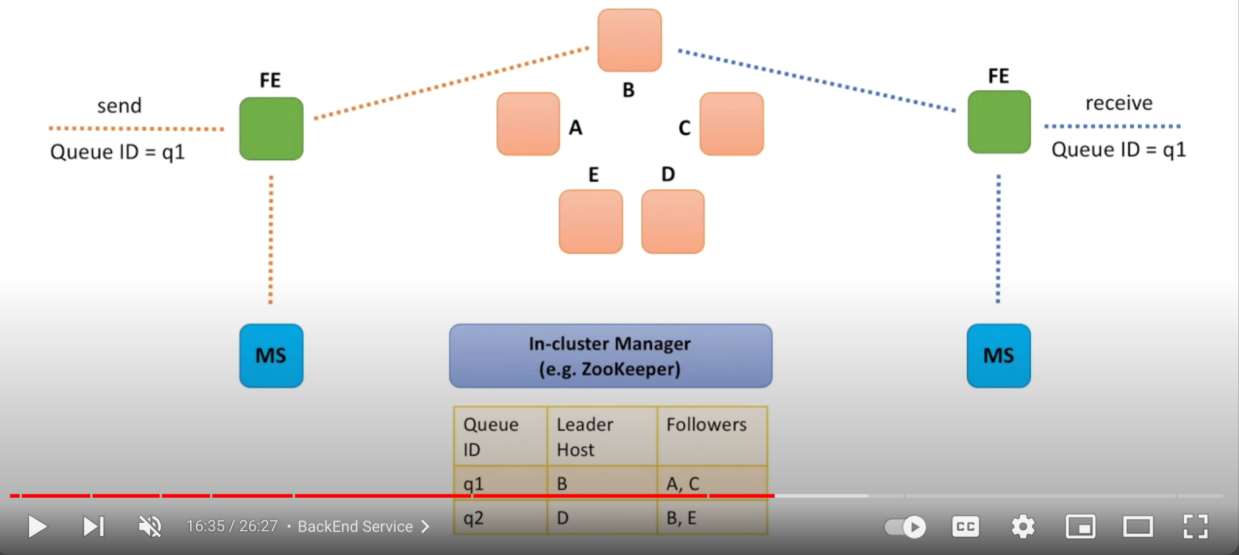
Answer: Replicate within a group of hosts

Answer: Metadata service



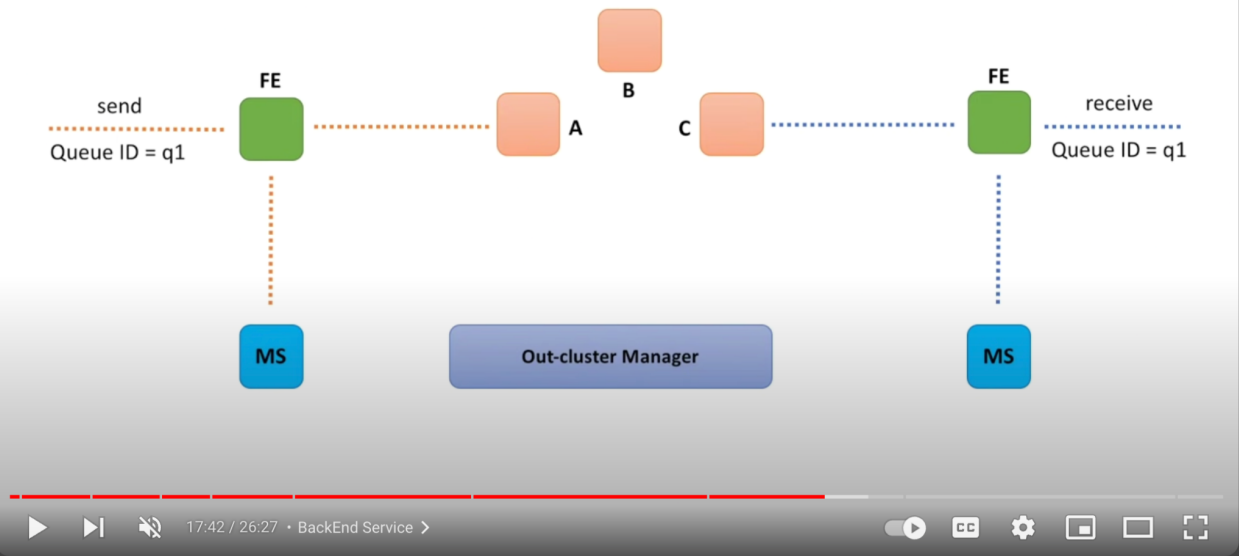
BackEnd Service

Option A: Leader-Follower relationship



BackEnd Service

Option B: Small cluster of independent hosts



BackEnd Service

In-cluster Manager

Manages queue assignment within the cluster

Maintains a list of hosts in the cluster

Monitors heartbeats from hosts

Deals with leader and follower failures

Splits queue between cluster nodes (partitioning)

Out-cluster Manager

Manages queue assignment among clusters

Maintains a list of clusters

Monitors each cluster health

Deals with overheated clusters

Splits queue between clusters



What else is important...

- Queue creation and deletion
- Message deletion
- Message replication
- Message delivery semantics
- Push vs. pull
- FIFO
- Security
- Monitoring

